





SVEUČILIŠTE U ZAGREBU
Fakultet elektrotehnike i računarstva

PARALELNO PROGRAMIRANJE

Poglavlje 02 - MPI



2.1 MPI STANDARD

- MPI je **standard** za paralelno programiranje temeljen na **razmjeni PORUKA** između **PROCESA.**
  - 1994: MPI 1.1, 1997: MPI 2.0, 2012: MPI 3.0, 2015: MPI 3.1, 2021: MPI 4.0, 2025: MPI 5.0
- Standard NIJE implementacija te je za njegovo korištenje potrebno instalirati neku **MPI implementaciju**, npr.:
  - MS-MPI (Microsoft MPI)
  - MPICH
  - Open MPI

MPI MODEL

- **SPMD** - svi procesi izvode isti program
  - Najčešća varijansta
  - Procesi mogu biti na jednom ili više računala
- **MPMD** - različiti programi za različite procese
  - rjeđa varijanta
  - može se simulirati uvrštenjem više funkcija u jedan SPMD program
- Načini komunikacije:
  - ***point-to-point*** – između **dva** određena procesa
  - ***collective*** – komunikacija unutar **grupe** procesa
  - *probe* – funkcije za **asinkronu** komunikaciju
  - ***communicator*** – podgrupe, mehanizam za razvoj modularnih paralelnih programa (mogućnost korištenja topologije mreže)

2.2 MPI FUNKCIJE

- Mehanizmi nužni za ostvarenje paralelnog programa u modelu razmjene poruka:
  - ***otkriti*** *ukupan* ***broj*** *procesa*
  - ***identificirati vlastiti*** *proces u grupi procesa*
  - ***poslati*** *poruku određenom procesu*
  - ***primiti*** *poruku (od određenog procesa)*
- MPI uključuje preko 150 funkcija, no većina funkcionalnosti može se postići sa mnogo manjim skupom
- Na predavanjima: **C/C++** (`#include "mpi.h"`)

OSNOVNE MPI FUNKCIJE

- U svakom programu:
  - na početku:
  - na kraju:

```
int MPI_Init(int *argc, char ***argv)
int MPI_Finalize()
```

- Ukupan **broj procesa** i **identifikacija** u grupi:

```
int MPI_Comm_size (MPI_Comm comm, int *size)

int MPI_Comm_rank (MPI_Comm comm, int *rank)
```

→ **komunikator** (grupa)

- skup procesa koji mogu komunicirati
- **MPI_COMM_WORLD** → globalna grupa uključuje **sve** procese

- *jedan proces može biti član više grupa, no u svakoj grupi proces ima posebni indeks*
- *korisnik može sam definirati dodatne grupe procesa (sve takve grupe su podskup početne globalne grupe)*

2.3 RAZMJENA PORUKA

- Osnovne funkcije slanja i primanja poruka:

```
int MPI_Send (void *buf, int count, MPI_Datatype datatype, int dest, int
tag, MPI_Comm comm)


int MPI_Recv (void *buf, int count, MPI_Datatype datatype, int source, int
tag, MPI_Comm comm, MPI_Status *status)
```

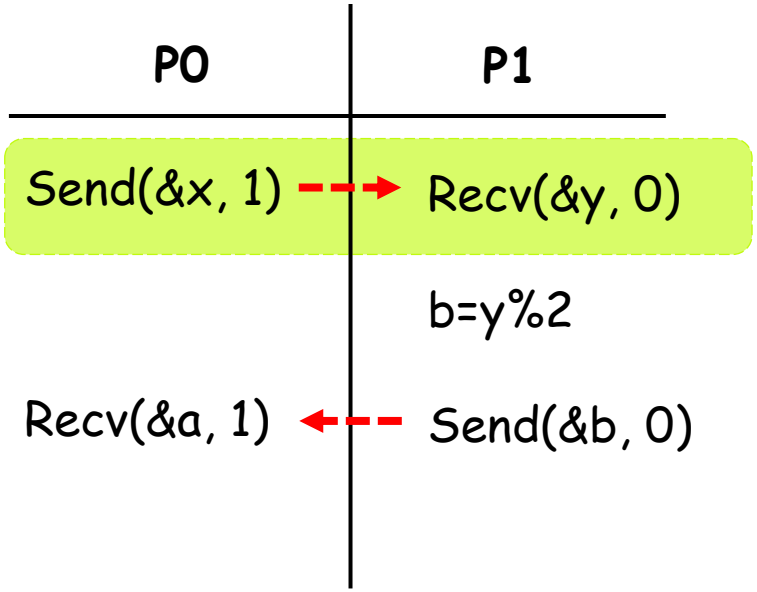
  - **buf** – početna adresa podataka u memoriji koji se šalju, odnosno primaju
  - **count** – broj jedinica podataka (duljina niza)
  - **datatype** – odgovarajući MPI tip podatka (`MPI_CHAR, MPI_INT, MPI_FLOAT`…)
  - **dest** i **source** – indeks (*rank*) procesa pošiljatelja i primatelja
  - **tag** – vrsta poruke (korisnik definira proizvoljno)
  - **comm** - komunikator (npr. `MPI_COMM_WORLD`)
  - **status** – struktura u kojoj će **nakon primitka** biti **zapisani podaci o poruci** (oznaka kao status.MPI_TAG, proces pošiljatelj kao status.MPI_SOURCE, itd.)

RAZMJENA PORUKA

- Uvjeti uspjeha komunikacije:
  - **indeksi** pošiljatelja i primatelja moraju **odgovarati**
  - mora biti naveden **isti komunikator**
  - **oznake vrste** poruke (*tag*) moraju biti iste
  - **memorijski prostor** primatelja mora biti dovoljno velik
- Poopćenje *primanja* poruke:
  - od **bilo kojeg** pošiljatelja – **`MPI_ANY_SOURCE`** kao *source* parametar
  - **za bilo koju oznaku** poruke – **`MPI_ANY_TAG`** kao *tag* parametar
  - konkretni podaci se mogu pročitati iz **statusa poruke**

SKRAĆENA SINTAKSA

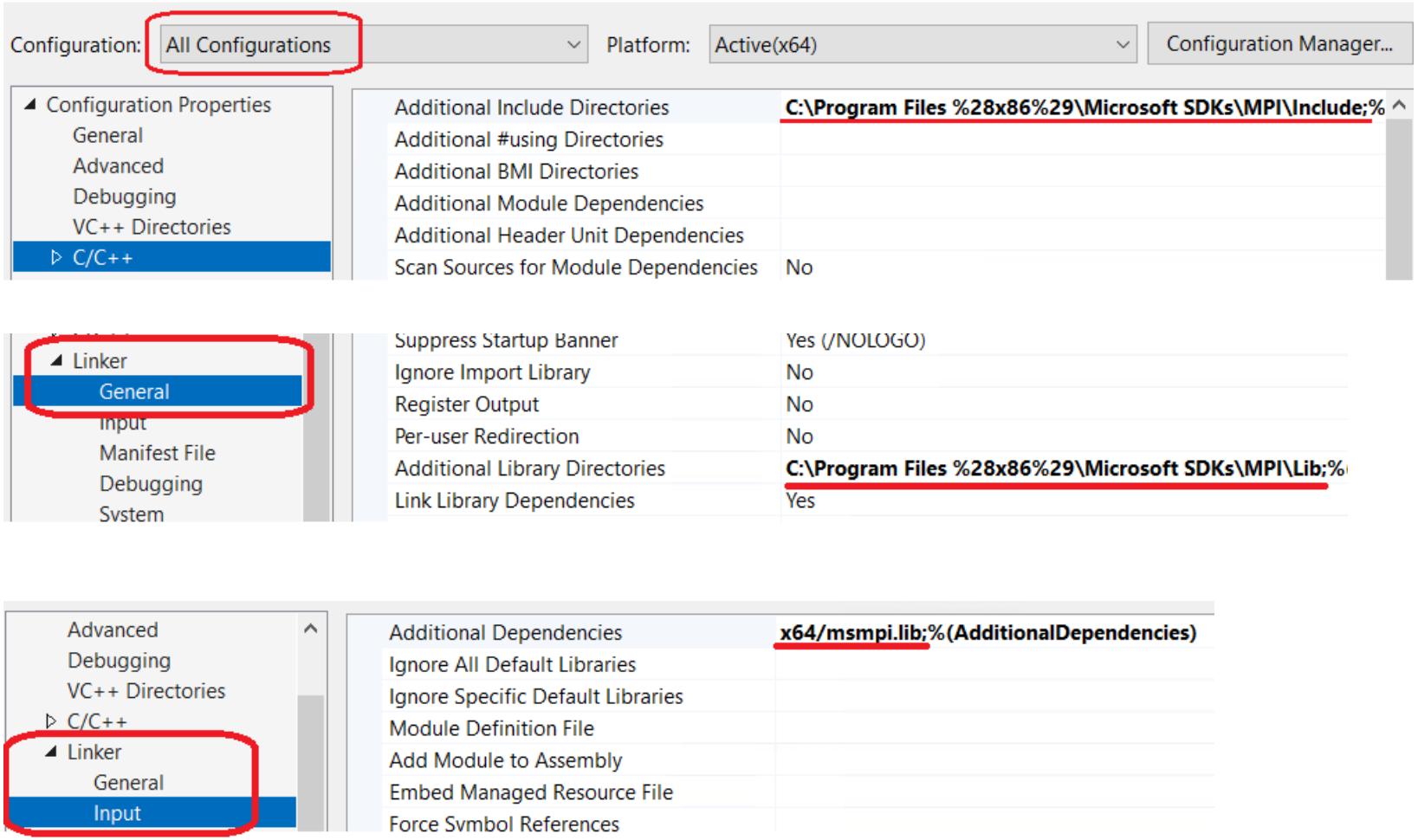
- Za rješavanje zadataka na ploči i na ispitu:
  - **Send (&data, dest)**
  - **Recv (&data, src)**
  - **Recv (&data, *)**



Send i Recv uvijek moraju biti upareni!

MS-MPI DEMO

VS POSTAVKE


Configuration: All Configurations
Platform: Active(x64)
Configuration Manager...
Configuration Properties
General
Advanced
Debugging
VC++ Directories
C/C++
Additional Include Directories
C:\Program Files %28x86%29\Microsoft SDKs\MPI\Include;%
Additional #using Directories
Additional BMI Directories
Additional Module Dependencies
Additional Header Unit Dependencies
Scan Sources for Module Dependencies
No
Linker
General
Input
Manifest File
Debugging
System
Suppress Startup Banner
Yes (/NOLOGO)
Ignore Import Library
No
Register Output
No
Per-user Redirection
No
Additional Library Directories
C:\Program Files %28x86%29\Microsoft SDKs\MPI\Lib;%
Link Library Dependencies
Yes
Advanced
Debugging
VC++ Directories
C/C++
Linker
General
Input
Additional Dependencies
x64/msmpi.lib;%(AdditionalDependencies)
Ignore All Default Libraries
Ignore Specific Default Libraries
Module Definition File
Add Module to Assembly
Embed Managed Resource File
Force Symbol References

HELLO WORLD

```
// inicijalizacija
MPI_Init(NULL, NULL);

// ukupan broj procesa
int world_size;
MPI_Comm_size(MPI_COMM_WORLD, &world_size);

// moj redni broj
int world_rank;
MPI_Comm_rank(MPI_COMM_WORLD, &world_rank);

// ime procesora (CPU)
char processor_name[MPI_MAX_PROCESSOR_NAME];
int name_len;
MPI_Get_processor_name(processor_name, &name_len);

// hello world poruka
printf("Hello world from processor %s, rank %d"
     " out of %d processors\n",
     processor_name, world_rank, world_size);

// kraj programa
MPI_Finalize();
```

Obavezno na početku!

Ukupno procesa

Vlastiti id

Obavezno na kraju!

HELLO WORLD

- mpiexec –n 5 MPIHello.exe



MPI RAZMJENA PORUKA

```
...
int number;

//slanje u lancu, proc id=0 počinje, zatim čeka što će dobiti od n-1-og
if (world_rank == 0) {
     number = 0;

     MPI_Send(&number, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);

     MPI_Recv(&number, 1, MPI_INT, world_size - 1, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);

}

else{
     MPI_Recv(&number, 1, MPI_INT, world_rank-1, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);

     number++;

     MPI_Send(&number, 1, MPI_INT, (world_rank + 1) % world_size, 0, MPI_COMM_WORLD);

}

...
```

MPI RAZMJENA PORUKA

- mpiexec –n 5 MPIComm.exe

```
Microsoft Windows [Version 10.0.17763.4737]
(c) 2018 Microsoft Corporation. All rights reserved.

E:\Nikolina\MPI\MPIComm\x64\Debug>mpiexec -n 5 MPIComm.exe
Proces 0 salje broj 0 procesu 1
Proces 0 prima broj 4 od procesa 4
Proces 3 prima broj 2 od procesa 2
Proces 3 salje broj 3 procesu 4
Proces 2 prima broj 1 od procesa 1
Proces 2 salje broj 2 procesu 3
Proces 1 prima broj 0 od procesa 0
Proces 1 salje broj 1 procesu 2
Proces 4 prima broj 3 od procesa 3
Proces 4 salje broj 4 procesu 0

E:\Nikolina\MPI\MPIComm\x64\Debug>_
```

MPI – NAČINI KOMUNIKACIJE

2.4 NAČINI KOMUNIKACIJE

- Dva osnovna načina ***point-to-point*** komunikacije:
  1. **BLOKIRAJUĆI** (*blocking*) – **povratak** iz funkcije znači da je ista **ZAVRŠILA**
  2. **NEBLOKIRAJUĆI** (*non-blocking*) – **povratak** iz funkcije je **trenutan**
     - korisnik mora naknadno **provjeriti uvjet završetka**
  - **definicija ZAVRŠETKA:** trenutak kada se može **sigurno pristupiti memorijskim lokacijama** korištenim u komunikaciji:
    - **slanje:** poslana varijabla se ponovno može upotrijebiti (npr. pisati)
      - *ne znači nužno da je poruka primljena (ovisi o implementaciji)!*
        a. *može* **čekati** dok poruka ne bude **isporučena** (odnosno dok proces primatelj ne pozove odgovarajući MPI_Recv - **sinkroni način, *synchronous mode***)
        b. *može* **kopirati** poruku u **međuspremnik** i odmah vratiti kontrolu pozivatelju – ovisno o veličini poruke i strategiji pojedine MPI implementacije (tzv. ***buffered mode***)
    - **primanje:** primljena varijabla se može upotrijebiti (čitati)
- **MPI_Send** i **MPI_Recv** su ***blokirajuće*** funkcije

NEBLOKIRAJUĆE FUNKCIJE

- Neblokirajuća komunikacija
  - pozivanje neblokirajuće funkcije → **povratak** iz funkcije **trenutan**, **završetak** za mora **provjeriti**
  - obavljanje posla koji ne uključuje podatke iz neblokirajućeg poziva
  - čekanje ili provjeravanje (u petlji) završetka funkcije

```
int MPI_Isend (void *buf, int count, MPI_Datatype datatype, int dest, int
tag, MPI_Comm comm, MPI_Request *request)

int MPI_Irecv (void *buf, int count, MPI_Datatype datatype, int source,
int tag, MPI_Comm comm, MPI_Request *request)
```

*parametar pomoću kojega se **ispituje završetak** funkcije*

PROVJERA ZAVRŠETKA FUNKCIJE/POSTOJANJA PORUKE

- Provjera završetka funkcije:

```
int MPI_Wait (MPI_Request *request, MPI_Status *status)
```

funkcija Wait ne vraća kontrolu pozivatelju dok se ne završi radnja identificirana parametrom request

```
int MPI_Test (MPI_Request *request, int *flag, MPI_Status *status)
```

funkcija Test vraća kontrolu odmah i upisuje vrijednost TRUE ili FALSE u parametar flag, ovisno o tome je li odgovarajuća neblokirajuća funkcija završila

- Provjerava pristiglih poruka:

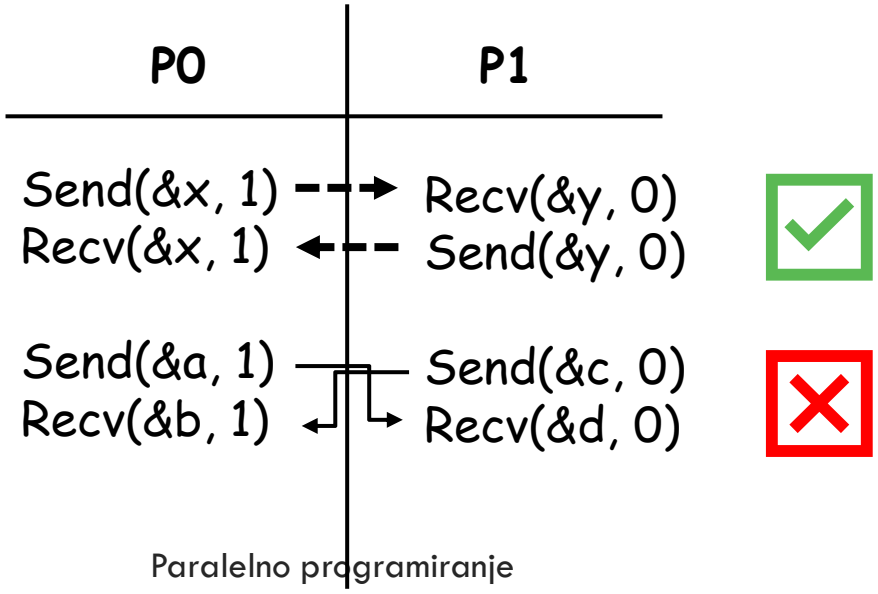
```
int MPI_Iprobe( int source, int tag, MPI_Comm com, int *flag,
MPI_Status *status )
```

neblokirajuća *funkcija koja provjerava* ***postoji li*** *(dolazna) poruka od izvora source sa oznakom tag, a rezultat sprema u parametar flag*

- nakon dospijeća informacije o poruci, izvor i oznaka poruke mogu se pročitati iz parametra ***status*** (status.MPI_SOURCE i status.MPI_TAG)
- poruka se tada može **primiti pozivom funkcije MPI_Recv**

DETERMINIZAM

- MPI programski model je u osnovi **nedeterministički**
  - ako **dva ili više** procesa **šalju** poruke **jednom** procesu, **redoslijed** primitka poruka **NIJE definiran**
  - **redoslijed** poruka **od jednog** procesa **prema drugom** je uvijek **očuvan** (*vrijedi samo za point-to-point komunikaciju*)
  - programer je odgovoran za postizanje determinizma – uporabom komunikatora te parametara *source* i *tag* za svaku poruku



2.5 GLOBALNA KOMUNIKACIJA

- Slanje/primanje podataka **1→N i N→1**
- Svojstva globalnih operacija:
  - ***SVI procesi u grupi moraju pozvati određenu funkciju***
  - sve funkcije su ***blokirajuće***
  - nema oznake poruke (parametar *tag*)
  - **količina** poslanih podataka mora odgovarati količini primljenih podataka!
- Operacije:
  - **Bcast**
  - **Reduce**
  - **Scatter**
  - **Gather**
  - **Barrier**

BROADCAST

- Slanje od jednog **svima** (u komunikatoru):

```
int MPI_Bcast (void *buffer, int count, MPI_Datatype datatype,
int root, MPI_Comm comm)
```

*izvorišni proces označen je parametrom **root** (indeks procesa)*

***buffer** se kod procesa root čita, dok se kod ostalih u njega upisuju podaci*

	buffer
P0	42
P1	
P2	
P3	

Bcast



	buffer
P0	42
P1	42
P2	42
P3	42

REDUCE

- Računanje jednog **rezultata na temelju podataka** od svih procesa:

```
int MPI_Reduce (void *sendbuf, void *recvbuf, int count, MPI_Datatype
datatype, MPI_Op op, int root, MPI_Comm comm)
```

skuplja podatke od svih procesa, uključujući i root proces, (s adrese sendbuf, duljine count) i nad njima provodi neku operaciju (definiranu s op parametrom) i rezultat sprema na adresu recvbuf na procesu root

neke predefinirane operacije: MPI_MAX, MPI_MIN, MPI_SUM, MPI_PROD (umnožak), MPI_LAND, MPI_BAND (logički i bitwise AND), itd....

	SB	RB
P0	2	
P1	3	
P2	4	
P3	5	

+_reduce



	SB	RB
P0	2	14
P1	3	
P2	4	
P3	5	

SCATTER

```
int MPI_Scatter (void *sendbuf, int sendcnt, MPI_Datatype sendtype, void
*recvbuf, int recvcnt, MPI_Datatype recvtype, int root, MPI_Comm cm)
```

raspodjeljuje po jedan dio niza (*sendbuf* na procesu *root*) svakom od procesa na adresu *recvbuf* (uključujući i *root* proces)

podebljani parametri se koriste samo na *root* procesu

vrsta podataka i broj elemenata koje *root* šalje *svakom* procesu određeni su sa *sendtype* i *sendcnt*; *recvtype* i *recvcnt* određuju vrstu i količinu podataka koje prima pojedini proces (*sendcnt* i *recvcnt* moraju biti jednaki!)

	SB	RB
P0	abcd	
P1		
P2		
P3		

SCATTER



	SB	RB
P0	abcd	a
P1		b
P2		c
P3		d

GATHER

```
int MPI_Gather (void *sendbuf, int sendcnt, MPI_Datatype sendtype, void
*recvbuf, int recvcnt, MPI_Datatype recvtype, int root, MPI_Comm cm)
```

prikuplja određeni dio niza od svih procesa (pohranjen u sendbuf na svim procesima) i sprema ih u recvbuf na root procesu

podebljani parametri se koriste samo na *root* procesu

	SB	RB
P0	A	
P1	B	
P2	C	
P3	D	

GATHER



	SB	RB
P0	A	ABCD
P1	B	
P2	C	
P3	D	

GLOBALNA SINKRONIZACIJA

- Globalna sinkronizacija za sve procese unutar grupe postiže se pozivom funkcije:

```
int MPI_Barrier (MPI_Comm comm)
```

- Za sve procese koji pozivaju funkciju vrijedi da niti jedan neće nastaviti sa radom dok je svi procesi ne pozovu

2.6 ISPRAVNOST PROGRAMA

- Standardom NIJE propisano hoće li se procesi prilikom globalne kom. sinkronizirati (ovisi o implementaciji)
  - NE smije se oslanjati na sinkronizaciju prilikom globalne komunikacije
  - MORA se pretpostavitii da globalna komunikacija *MOŽE* biti sinkonizirajuća

```
switch(rank)
{   case 0:
        MPI_Bcast (buf1, count, type, 0, comm);
        MPI_Bcast (buf2, count, type, 1, comm);
    break;
    case 1:
        MPI_Bcast (buf2, count, type, 1, comm);
        MPI_Bcast (buf1, count, type, 0, comm);
    break;
}
```



```
switch(rank)
{   case 0:
        MPI_Bcast (buf1, count, type, 0, comm);
        MPI_Send (buf2, count, type, 1, tag, comm);
    break;
    case 1:
        MPI_Recv (buf2, count, type, 0, tag, comm, status);
        MPI_Bcast (buf1, count, type, 0, comm);
    break;
}
```



```
switch(rank)
{   case 0:
        MPI_Bcast (buf1, count, type, 0, comm);
        MPI_Send (buf2, count, type, 1, tag, comm);
    break;
    case 1:
        MPI_Recv (buf2, count, type, MPI_ANY_SOURCE, tag, comm, status);
        MPI_Bcast (buf1, count, type, 0, comm);
        MPI_Recv (buf2, count, type, MPI_ANY_SOURCE, tag, comm, status);
    break;
    case 2:
        MPI_Send (buf2, count, type, 1, tag, comm);
        MPI_Bcast (buf1, count, type, 0, comm);
    break;
}
```

ZADACI

ZADATAK 1

Korištenjem MPI funkcija Send i Recv (skraćena sintaksa) napišite niz instrukcija koji će **sve elemente zadane kružne liste postaviti na srednju vrijednost toga i dvaju susjednih elemenata** (indeksi i, i+1, i-1; posljednji element povezan je s prvim i obrnuto).

Svaki MPI proces ima u lokalnoj memoriji samo jedan element liste koji je realna vrijednost. **Broj procesa je N**, svaki proces ima **redni broj ID**. Program treba jamčiti ispravnost rada bez obzira na veličinu poruka (ne smije doći do potpunog zastoja zbog redoslijeda slanja i primanja)!

ZADATAK 2

U jednom trenutku rada paralelnog programa u n procesa se nalaze neki podaci. Potrebno je odrediti **najveći element od svih n podataka** i tu informaciju (vrijednost najvećega) proslijediti svim procesima. Napisati algoritam koji će obaviti taj zadatak pomoću MPI funkcija MPI_Send i MPI_Recv.

Uputa: slanje i primanje poruka obaviti u obliku **lanca u dva prolaza** (s lijeva na desno te potom s desna na lijevo po svim procesima). Kod poziva MPI funkcija navesti samo 'bitne' parametre (npr. MPI_Send(&varijabla,_,_,odrediste,_,_);).

ZADATAK 3

Korištenjem MPI funkcija Send i Recv (skraćena sintaksa) napisati odsječak programa (proizvoljne složenosti) koji će za N procesa **ostvariti funkciju MPI_Barrier**, tj. postići da svi procesi moraju doći do istog odsječka prije nego bilo koji proces može nastaviti s izvođenjem. (U svakom procesu varijabla ID je indeks, a varijabla N ukupni broj procesa.)

ZADATAK 4

Zadan je MPI program (na slici). Svi procesi imaju lokalne varijable a, b i c, a ID je indeks pojedinog procesa. Koje vrijednosti će imati varijabla c za svaki proces na kraju izvođenja? Navedite **sve mogućnosti i skicirajte redoslijed izvođenja** MPI operacija za svaki proces.

```
// Proces 1, ID = 1
MPI_Send(&ID, _, _, 2, _, _);
MPI_Recv(&a, _, _, MPI_ANY_SOURCE, _, _);
MPI_Send(&a, _, _, 3, _, _);
MPI_Recv(&b, _, _, MPI_ANY_SOURCE, _, _);
c = 2*a + b;
```

```
// Proces 2, ID = 2
MPI_Recv(&a, _, _, MPI_ANY_SOURCE, _, _);
MPI_Recv(&b, _, _, MPI_ANY_SOURCE, _, _);
c = 2*a + b;
MPI_Send(&c, _, _, 1, _, _);
MPI_Send(&ID, _, _, 3, _, _);
```

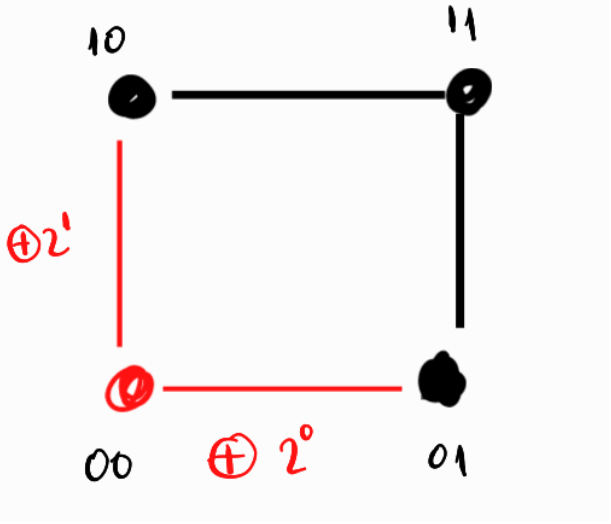
```
// Proces 3, ID = 3
MPI_Send(&ID, _, _, 2, _, _);
MPI_Recv(&a, _, _, MPI_ANY_SOURCE, _, _);
MPI_Recv(&b, _, _, MPI_ANY_SOURCE, _, _);
MPI_Send(&a, _, _, 1, _, _);
c = 2*a + b;
```

ZADATAK 5

U MPI programu svaki proces ima lokalnu vrijednost u varijabli x. Korištenjem MPI funkcija Send i Recv (skraćena sintaksa) napisati odsječak programa **logaritamske složenosti** (po pitanju broja poslanih poruka) koji će za N procesa izračunati **minimum svih lokalnih vrijednosti**, tako **da svi procesi znaju rezultat.** (U svakom procesu varijabla ID je indeks, a varijabla N ukupni broj procesa.)

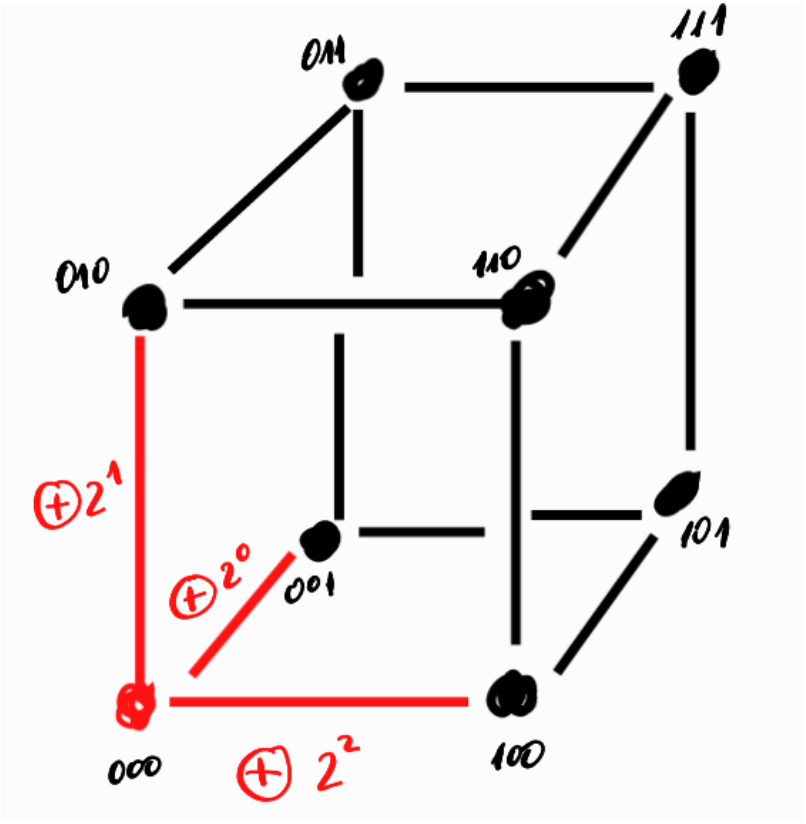
HIPERKOCKA

- n=2

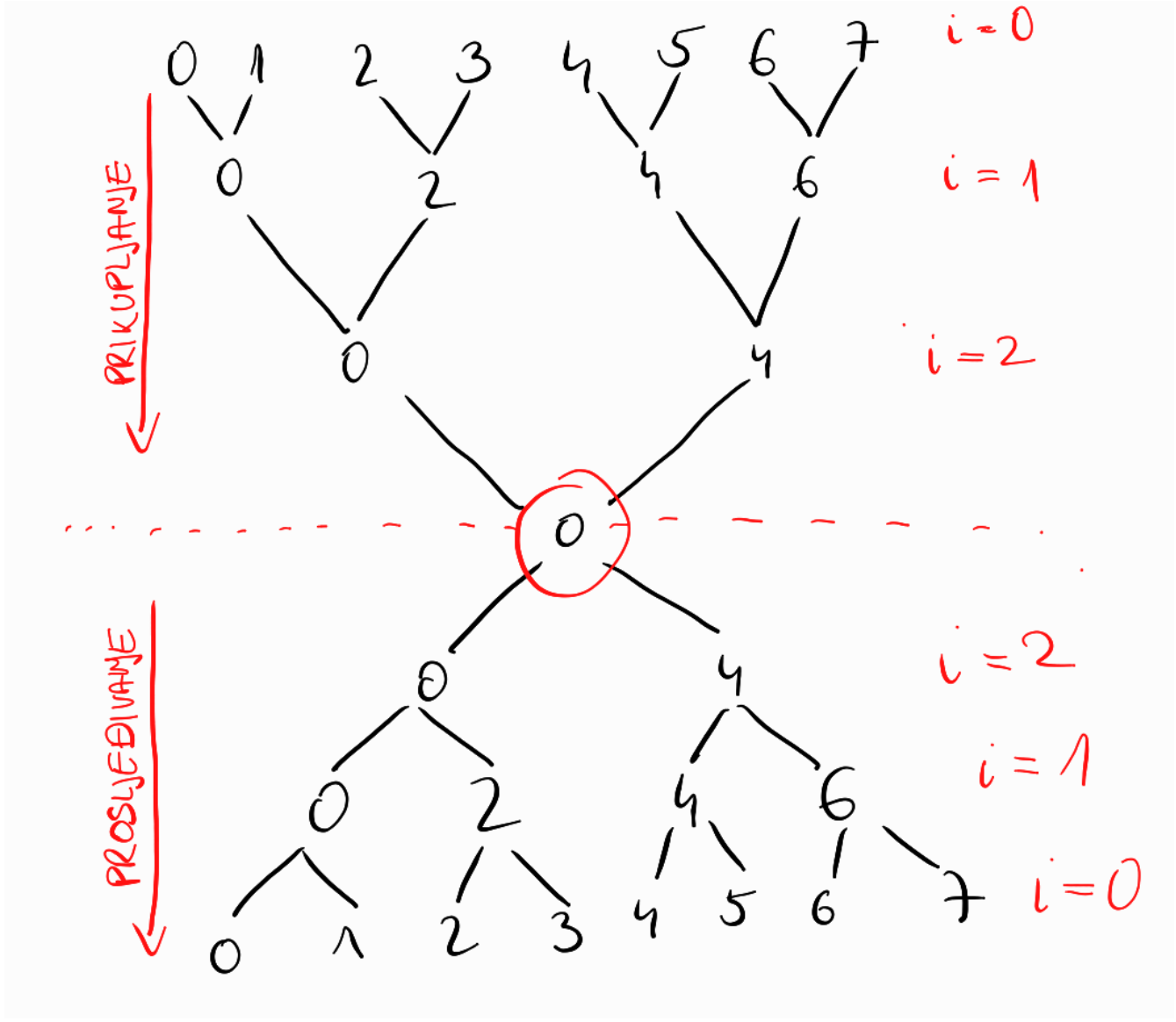


Grayev kod!

- n=3



BINARNO STABLO



ZADATAK 6

U MPI programu u nekom trenutku **pojavio se promatrani događaj** unutar jednog MPI procesa. Svi procesi znaju da se događaj pojavio, ali nijedan proces (osim dotičnog, izvorišnog procesa) ne zna unutar kojeg procesa se pojavio događaj (tj. koji je proces izvorišni). Korištenjem MPI funkcija Send i Recv (skraćena sintaksa) napisati odsječak programa **logaritamske složenosti** (po pitanju broja poslanih poruka) koji će za N procesa omogućiti da **svi procesi saznaju indeks izvorišnog procesa**. (U svakom procesu varijabla ID je indeks, a varijabla N ukupni broj procesa)

ZADATAK 7

U MPI programu u nekom trenutku svih N procesa treba obaviti kritični odsječak. Svaki proces zna svoj **redni broj ulaska u K.O.,** ali ne zna redne brojeve ostalih procesa. Korištenjem MPI funkcija Send i Recv (skraćena sintaksa) napisati odsječak programa **logaritamske složenosti** (po pitanju broja poslanih poruka) koji će omogućiti da **svaki proces sazna indeks svog neposrednog prethodnika i sljedbenika** (pozivanje K.O. nije potrebno prikazati). U svakom procesu varijabla ID je indeks procesa, a varijabla RBR redni broj ulaska u K.O.